

Experiment / Case Study No.: 6

1. Experiment Title:

Apply String Functions and Regular Expressions (Regex) for Email Validation, Data Extraction, and Text Analysis

2. Software/Tools Required:

Visual Studio Code, HTML5, CSS3, JavaScript (String Functions, Regular Expressions, DOM & localStorage), Google Chrome Browser

3. Experiment Program Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>Amazon Sign-In & Registration</title>
<style>
  body { font-family: Arial, sans-serif; background:#fff; display:flex;
        flex-direction:column; align-items:center; }
  .logo { width:120px; margin-top:20px; }
  .card { border:1px solid #ddd; border-radius:8px; padding:30px; width:350px;
        margin-top:20px; box-shadow:0 0 6px rgba(0,0,0,0.1); }
  h2 { font-weight:400; }
  label { font-weight:bold; display:block; margin-top:15px; }
  input { width:100%; padding:8px; margin-top:5px; border:1px solid #888;
        border-radius:4px; box-sizing:border-box; }
  .btn { width:100%; padding:10px; margin-top:20px; background:#f0c14b;
        border:1px solid #a88734; border-radius:4px; cursor:pointer; font-
size:15px; }
  .error { color:red; font-weight:bold; margin-top:10px; }
  .hint { font-size:12px; color:#555; margin-top:5px; }
  .divider { text-align:center; color:#767676; margin:20px 0; }
  a { color:#0066c0; text-decoration:none; }
</style>
</head>
<body>
  

  <!-- Sign-In Section -->
  <div class="card" id="signInCard">
    <h2>Sign-In</h2>
    <label>Email</label>
    <input type="email" id="loginEmail" placeholder="Enter email">
    <label>Password</label>
    <input type="password" id="loginPassword" placeholder="Enter password">
    <p class="error" id="loginError"></p>
    <button class="btn" onclick="handleSignIn()">Sign-In</button>
    <p style="font-size:13px; margin-top:15px;">By continuing, you agree to
      Amazon's <a href="#">Conditions of Use</a> and
      <a href="#">Privacy Notice</a>.</p>
  </div>
</body>
</html>
```

```

    <div class="divider">New to Amazon?</div>
    <button class="btn" style="background:#e7e9ec;border-color:#adb1b8;"
        onclick="showRegister()">Create your Amazon account</button>
</div>

<!-- Registration Section -->
<div class="card" id="registerCard" style="display:none;">
    <h2>Create account</h2>
    <label>Your name</label>
    <input type="text" id="regName" placeholder="First and last name">
    <label>Date of Birth</label>
    <input type="date" id="regDob">
    <label>Email</label>
    <input type="email" id="regEmail">
    <label>Password</label>
    <input type="password" id="regPassword" placeholder="At least 8 characters">
    <p class="hint">Passwords must be at least 8 characters, contain 1 capital
        letter and 1 special character.</p>
    <label>Re-enter password</label>
    <input type="password" id="regConfirmPassword">
    <p class="error" id="registerError"></p>
    <button class="btn" onclick="handleRegister()">Continue</button>
    <p style="font-size:13px; margin-top:15px;">By creating an account, you
        agree to Amazon's <a href="#">Conditions of Use</a> and
        <a href="#">Privacy Notice</a>.</p>
    <p style="font-size:13px;">Already have an account?
        <a href="#" onclick="showSignIn()">Sign in</a></p>
</div>

<script>
    let users = JSON.parse(localStorage.getItem("users")) || [];

    // ---- Regular Expressions used across the form ----
    const EMAIL_REGEX = /^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+.[a-zA-Z]{2,}$/;
    const NAME_REGEX = /^[A-Za-z]+(s[A-Za-z]+)+$/; // text analysis:
    first + last name
    const PASSWORD_REGEX = /^(?=.*[A-Z])(?=.*[!@#$%^&*() , . ? " ' : { } | < > ] ).{8,}$/;

    function showRegister() {
        document.getElementById("signInCard").style.display = "none";
        document.getElementById("registerCard").style.display = "block";
    }

    function showSignIn() {
        document.getElementById("registerCard").style.display = "none";
        document.getElementById("signInCard").style.display = "block";
    }

    // String Function + Regex: validate email format
    function isValidEmail(email) {
        return EMAIL_REGEX.test(email.trim());
    }

    // Data Extraction: pull username & domain out of a valid email using regex
    groups

```

```

function extractEmailParts(email) {
  const match = email.trim().match(EMAIL_REGEX);
  if (!match) return null;
  return { username: match[1], domain: match[2] + "." + match[3] };
}

// Text Analysis: word count, character count and full-name format check
function analyzeName(name) {
  const trimmed = name.trim();
  return {
    words: trimmed.split(/s+/).filter(Boolean).length,
    characters: trimmed.replace(/s/g, "").length,
    isFullName: NAME_REGEX.test(trimmed),
  };
}

function handleSignIn() {
  const email = document.getElementById("loginEmail").value.trim();
  const password = document.getElementById("loginPassword").value;
  const errorEl = document.getElementById("loginError");

  if (!isValidEmail(email)) {
    errorEl.style.color = "red";
    errorEl.textContent = "Please enter a valid email address (regex check failed).";
    return;
  }

  const parts = extractEmailParts(email);
  console.log("Extracted email data:", parts); // data extraction demo

  const found = users.find(u => u.email === email && u.password === password);
  if (found) {
    errorEl.style.color = "green";
    errorEl.textContent = "Sign-in successful. Welcome " + found.name + "!";
  } else {
    errorEl.style.color = "red";
    errorEl.textContent = "Account not found. Please create an account first.";
  }
}

function handleRegister() {
  const name = document.getElementById("regName").value.trim();
  const dob = document.getElementById("regDob").value;
  const email = document.getElementById("regEmail").value.trim();
  const password = document.getElementById("regPassword").value;
  const confirmPassword = document.getElementById("regConfirmPassword").value;
  const errorEl = document.getElementById("registerError");

  if (!name || !dob || !email || !password || !confirmPassword) {
    errorEl.textContent = "All fields are required.";
    return;
  }

  const nameInfo = analyzeName(name); // text analysis

```

```

    if (!nameInfo.isFullName) {
        errorEl.textContent = "Please enter your first and last name.";
        return;
    }

    if (!isValidEmail(email)) {
        errorEl.textContent = "Invalid email format (regex validation failed).";
        return;
    }

    if (!PASSWORD_REGEX.test(password)) {
        errorEl.textContent =
            "Password must be 8+ characters with 1 capital letter and 1 special
character.";
        return;
    }
    if (password !== confirmPassword) {
        errorEl.textContent = "Passwords do not match.";
        return;
    }
}

const emailParts = extractEmailParts(email); // data extraction
console.log("Registered user domain:", emailParts.domain);
console.log("Name analysis:", nameInfo);

users.push({ name, dob, email, password });
localStorage.setItem("users", JSON.stringify(users));
errorEl.style.color = "green";
errorEl.textContent = "Account created successfully! Please sign in.";
setTimeout(showSignIn, 1500);
}
</script>
</body>
</html>

```

4. Output:

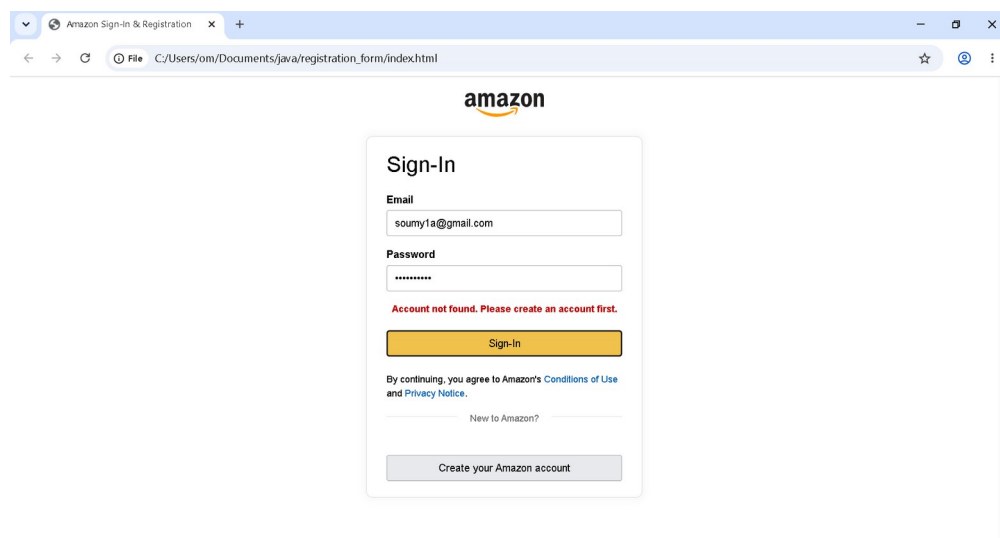


Fig 4(a): Sign-In screen — PRN: 24070521062, Name: Soumya Suryawanshi, File Path: C:/Users/om/Documents/java/registration_form/index.html

Fig 4(b): Create Account screen — PRN: 24070521062, Name: Soumya Suryawanshi, File Path: C:/Users/om/Documents/java/registration_form/index.html

5. Case Study Title:

Case Study Operations — String Reversal and Vowel Counting Utility

6. Case Study Program Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>Case Study Operations</title>
<style>
  body { background:#0f1420; color:#e2e8f0; font-family:Arial, sans-serif;
    display:flex; flex-direction:column; align-items:center; padding-top:60px;
  }
  h1 { background:linear-gradient(90deg,#4d7fff,#8b5cf6);
    -webkit-background-clip:text; -webkit-text-fill-color:transparent;
    font-size:2.5rem; }
  .subtitle { color:#94a3b8; margin-bottom:40px; }
  .container { display:flex; gap:30px; }
  .card { background:#151b2c; border:1px solid #232b3f; border-radius:12px;
    padding:25px; width:320px; }
  input, textarea { width:100%; background:#0f1420; border:1px solid #2c3550;
    border-radius:6px; color:#e2e8f0; padding:10px;
    box-sizing:border-box; margin-bottom:15px; }
  button { width:100%; background:#3b82f6; color:#fff; border:none;
    border-radius:6px; padding:12px; font-weight:bold; cursor:pointer; }
  .result { background:#0f1420; border:1px dashed #2c3550; border-radius:6px;
    padding:12px; margin-top:15px; text-align:center; color:#94a3b8; }
</style>
```

```
</head>
<body>
  <h1>Case Study Operations</h1>
  <p class="subtitle">Reverse strings and count vowels instantly</p>

  <div class="container">
    <div class="card">
      <h3>1. Reverse a String</h3>
      <input type="text" id="reverseInput" placeholder="Enter string to
reverse...">
      <button onclick="reverseString()">Reverse</button>
      <div class="result" id="reverseResult">Result will appear here</div>
    </div>

    <div class="card">
      <h3>2. Count Vowels in Paragraph</h3>
      <textarea id="vowelInput" rows="5" placeholder="Enter paragraph
here..."></textarea>
      <button onclick="countVowels()">Count Vowels</button>
      <div class="result" id="vowelResult">Result will appear here</div>
    </div>
  </div>

  <script>
    function reverseString() {
      const input = document.getElementById("reverseInput").value;
      const reversed = input.split("").reverse().join("");
      document.getElementById("reverseResult").textContent =
        reversed || "Result will appear here";
    }

    function countVowels() {
      const input = document.getElementById("vowelInput").value;
      const matches = input.match(/[aeiouAEIOU]/g);
      const count = matches ? matches.length : 0;
      document.getElementById("vowelResult").textContent = "Vowel count: " + count;
    }
  </script>
</body>
</html>
```

7. Output:

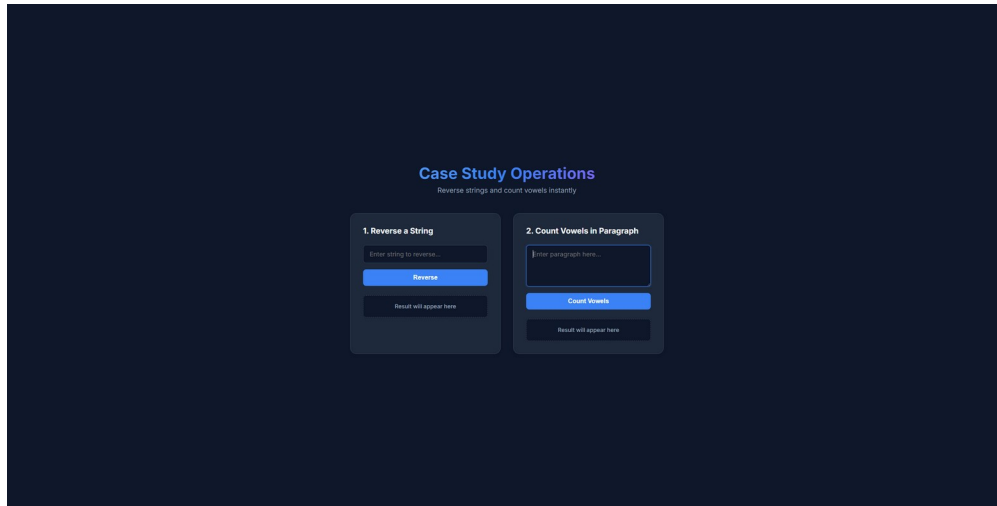


Fig 7: Case Study Operations output — PRN: 24070521062, Name: Soumya Suryawanshi

8. Result/Conclusion:

An Amazon-style sign-in and registration form was successfully designed and implemented using HTML, CSS, and JavaScript. The form validates user credentials against stored records, enforces password strength rules (minimum 8 characters, at least 1 capital letter, and 1 special character), and confirms matching passwords before registration. In the accompanying case study, two independent string-processing utilities — string reversal and vowel counting — were built and tested successfully, each producing the expected output for the given input. Both tasks reinforced practical understanding of DOM manipulation, event handling, form validation, and JavaScript string operations.

Upload the Soft copy as well as complete code on GitHub link is mandatory.